v 1.10

Tema 3

Herencia y polimorfismo con Java

Programación Orientada a Objetos

José Francisco Vélez Serrano

Universidad Rey Juan Carlos

Escuela Técnica Superior de Ingeniería Informática

Índice

- La herencia de clases
- Polimorfismo dinámico
- Clases abstractas e interfaces

La herencia de clases

Java permite la herencia simple de clases, que básicamente permite que una clase copie de otra su interfaz, su comportamiento y que añada nuevo comportamiento en forma de código.

Definición de herencia en Java



La herencia de clases posibilita continuar desarrollando una clase base (padre o super clase) en una nueva clase derivada (clase hija o sub-clase).

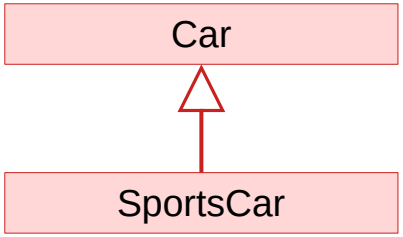
En Java se usa la palabra reservada extends para indicar que una clase deriva de otra.

```
[Modificador de clase] class <identificador> [parámetros] [herencia] [excepciones] {
    [método|propiedad|clase|bloque inicialización]*
}

[herencia] ::= [extends <super_clase>]
```

```
class Car {
}

class SportsCar extends Car {
}
```



¿Qué permite la herencia?



En general, en una clase derivada se puede:

- Definir nuevos miembros que se añaden a los ya existentes en las clases base.
- Invocar miembros de las clases base.

Estas posibilidades están limitadas por algunos modificadores usados en las clases base. La principal limitación es que miembros privados en la clase base no son visibles en las clases derivadas.

La herencia se usa para añadir funcionalidad



La herencia suele usarse para añadir funcionalidad a una clase de la que se hereda.

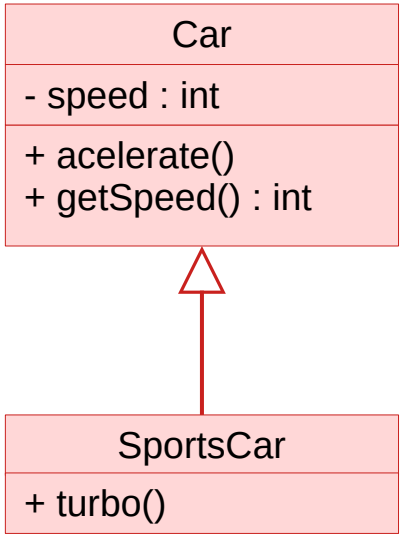
Por ejemplo: se hereda de una clase Stack y se añade un método toArray que devuelve los elementos de la Stack en un array.

En este caso la herencia evita duplicar código (copiar y pegar) para crear una clase muy similar a otra que ya tenemos.

También evita modificar clases que funcionan correctamente, y que ya son usadas por otros clientes, cuando se les quiere añadir nuevas funcionalidades.

La herencia de clases

"Herencia" vs "copiar y pegar código"




```java
//Clase base, padre o super clase
public class Car {
  private int speed;

  public void getSpeed() {
     return speed;
  }

  public void acelerate() {
     speed++;
  }
}
```

```java
//Clase derivada o clase hija
public class SportsCar extends Car {

  public void turbo() {
    for (int c = 0; c < 10; c++)
      acelerate();
  }
}
```

≈

```java
public class SportsCar {
  private int speed;

  public void getSpeed() {
     return speed;
  }

  public void acelerate() {
     speed++;
  }

  public void turbo() {
    for (int c = 0; c < 10; c++)
      acelerate();
  }
}
```

Se puede invocar en la clase hija

No se puede invocar en la hija

Ejemplo de herencia

```java
//Clase base, padre o super clase
public class Car {
  private int speed;

  public void getSpeed() {
     return speed;
  }

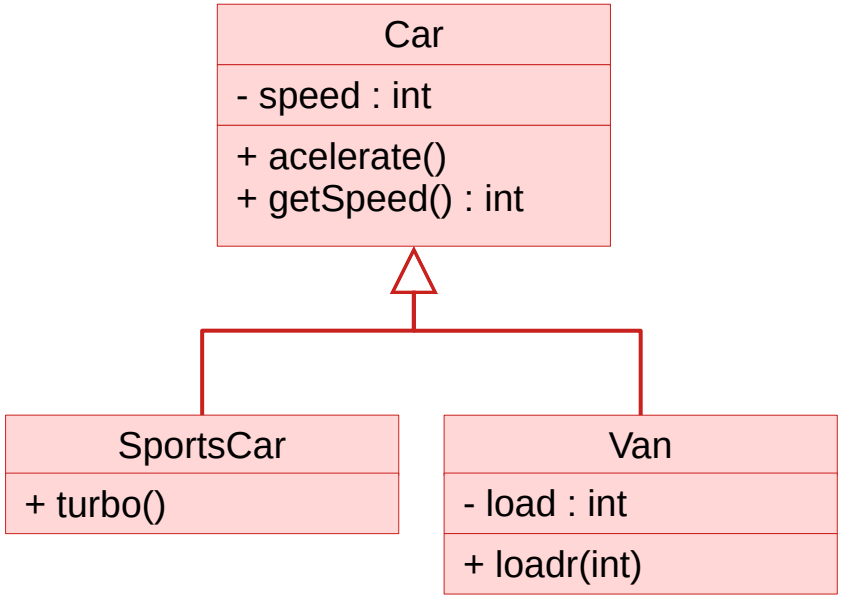
  public void acelerate() {
     speed++;
  }
}
```

```java
//Clase derivada o clase hija
public class SportsCar extends Car {

  public void turbo() {
    for (int c = 0; c < 10; c++)
      acelerate();
  }
}
```

```java
//Clase derivada o clase hija
public class Van extends Car {
  private int load;

  public void loadr(int load) {
    this.load = load;
  }
}
```



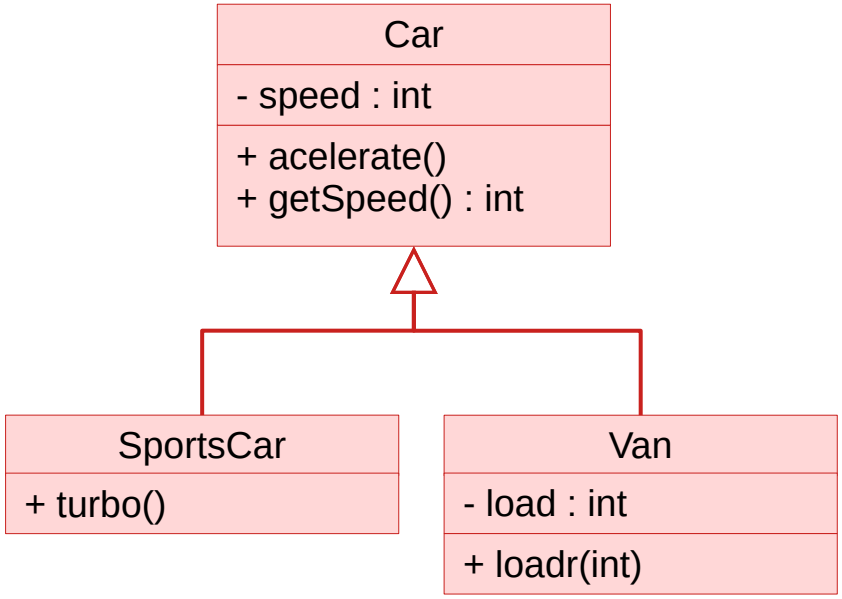
Car define los métodos acelerate y getSpeed, por tanto SportsCar y Van los tienen. SportsCar añade turbo, mientras que Van añade loadr.

Cada objeto tiene sus propias propiedades

```
public static void main() {
  SportsCar d = new SportsCar();
  Van f = new Van();

  d.acelerate();
  d.acelerate();
  f.acelerate()

  System.out.println(d.getSpeed());
  System.out.println(f.getSpeed());
}
```



Cada objeto de la clase Car tendrá su propia propiedad speed con un valor independiente.

Java tiene una única jerarquía de clases



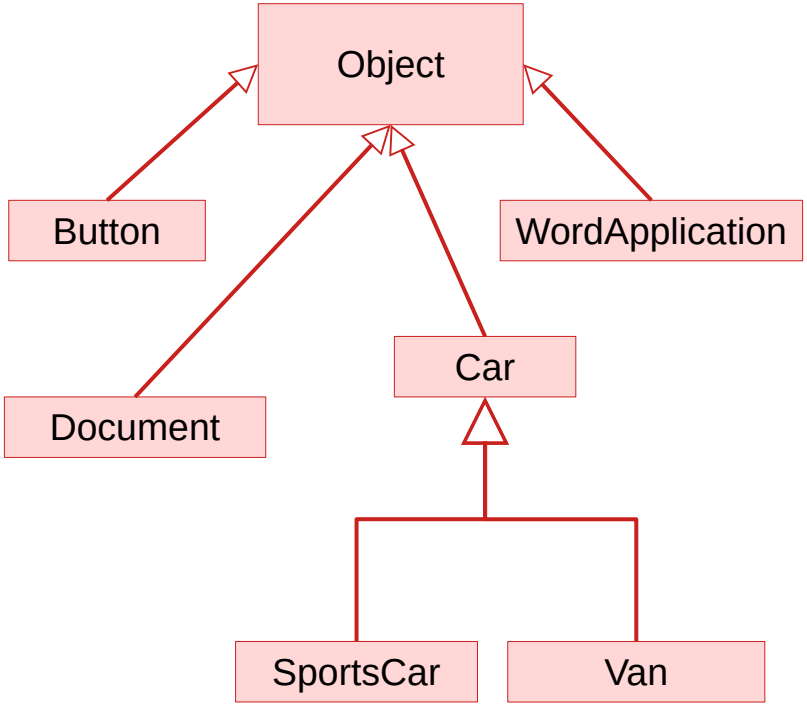
En Java cada clase deriva de otra clase.

En Java solo hay un árbol de herencia.

La clase `Object` está en la raíz del árbol de herencia de Java.

Cuando no se declara de qué clase deriva una clase, Java la hace derivar de la clase `Object`.

En Java una clase no puede derivar de dos o más clases.



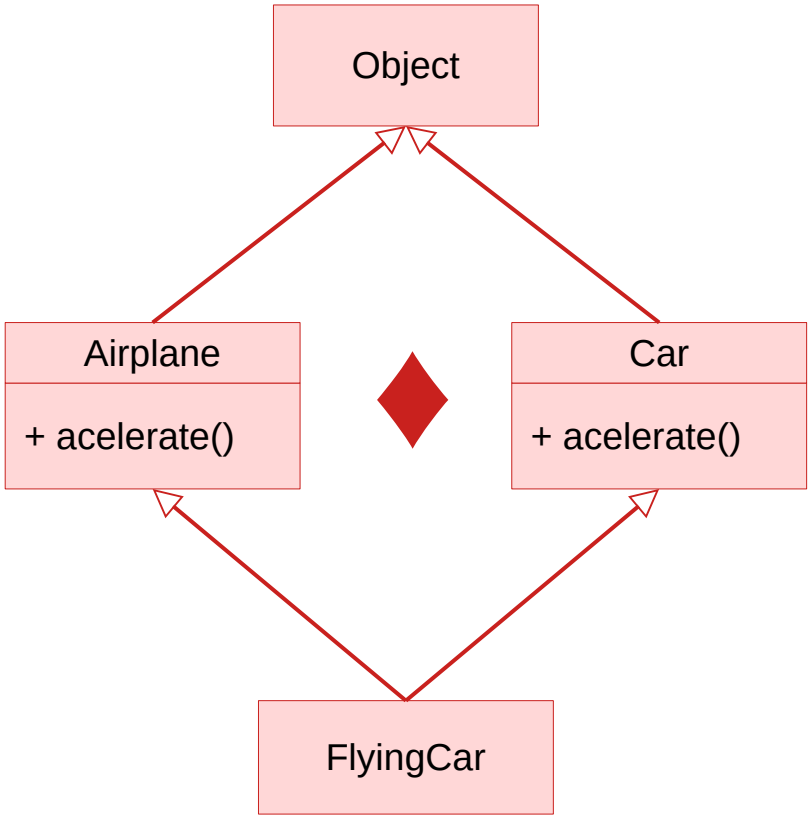
Herencia múltiple y el problema del diamante



Si una clase hereda de dos o más clases aparece al problema del diamante.

Dicho problema se deriva de la ambigüedad resultante al invocar un método definido en dos puntos de una jerarquía. ¿Qué método se ejecutaría cuando se le pide acelerate a un CarVolador? ¿Quién invoca antes al constructor de `Object`?

Java no permite la herencia múltiple de clases, evitando este problema. Otros lenguajes (C++, Python...) sí la permiten, y definen complejas políticas para resolver la ambigüedad.



Modificador de control de acceso protected

Un miembro protected (# en UML) es accesible desde:

- la propia clase
- clases que hereden de ella
- clases definidas en el mismo paquete.

```
public class Car {
  protected int speed;

  public void acelerate() {
     speed++;
  }
}

public class SportsCar extends Car {

  public void turbo() {
    for (int c = 0; c < 10; c++)
      acelerate();
  }

  public void brake() {
    speed = 0;
  }
}
```

Resumen de control de acceso



	Visibilidad			
Modificador	Clase	Paquete	Subclase	Cualquiera
public	Sí	Sí	Sí	Sí
protected	Sí	Sí	Sí	No
sin modificador	Sí	Sí	No	No
private	Sí	No	No	No

Sobrescritura de métodos y super

Cuando en una clase derivada se redefine un método ya definido en una clase base se dice que se está sobreescribiendo el método.

La declaración del método que sobrescribe debe ser igual a la del método sobrescrito añadiendo el tag @override.

Desde una clase derivada siempre se puede invocar al método de la clase base usando la propiedad privada `super`.

Los clientes de una clase derivada no ven los miembros sobrescritos de la clase base, solo ven la versión proporcionada por la clase derivada.

```
public class Car {
  private int speed;

  public void acelerate() {
     speed++;
  }

  public int getSpeed() {
     return speed;
  }
}

public class SportsCar extends Car {

  @override
  public void acelerate() {
    super.acelerate();
    super.acelerate();
    turbo = true;
  }
}
...
externalMethod() {
  SportsCar f = new SportsCar();
  f.acelerate();
  System.out.println(f.getSpeed());
}
```

2

Métodos final

Los métodos definidos como final no pueden cambiar su implementación en clases derivadas. No pueden ser sobreloaddos.

El uso de final tiene dos objetivos:

- Fijar el diseño, pues fijar razones de diseño al impedir cambios.
- Aumentar la eficiencia, pues permiten ligadura estática de las funciones lo que redunda en mayor speed.

```
public class Car {
  private int speed;

  public void acelerate() {
     speed++;
  }

  final public int getSpeed() {
     return speed;
  }
}

public class SportsCar extends Car {
  boolean turbo = false;

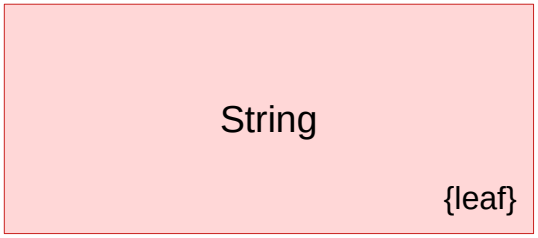
  @override
  public void acelerate() {
    super.acelerate();
    super.acelerate();
    turbo = true;
  }
}
```

Clases finales

De una clase final no se puede heredar para crear una nueva clase.

Se suele usar para reforzar decisiones de diseño, como por ejemplo impedir que se derive de una clase inmutable para hacerla mutable.

```
final class String {
     char      charAt(int index){...
     int       compareTo(String anotherString){...
     String    concat(String str){...
     boolean   contains(CharSequence s){...
     boolean   equals(Object anObject){...
     byte[]    getBytes(){...
     int       indexOf(int ch){...
     int       lastIndexOf(int ch){...
     int       length(){...
     boolean   matches(String regex){...
     String    replace(char oldChar, char newChar){...
     String    substring(int beginIndex, int endIndex){...
     String    toLowerCase(){...
     String    trim(){...
     static    String    valueOf(double d){...
     ...
}
```



La herencia y los constructores



En Java, una clase siempre debe declarar explícitamente los constructores que desea.

Los constructores de una clase padre no son visibles para los clientes de una clase hija.

Desde el constructor de una clase derivada se puede llamar al constructor de la superclase utilizando super.

Si no se llama al constructor de la clase padre, al definir un constructor en una clase derivada, Java llama al constructor sin parámetros de la superclase (si la superclase no tuviese constructor sin parámetros no comStackría).

```
public class Car {
  private int speed;

  public Car(int speed) {
    this.speed = speed;
  }

  public void acelerate() {
     speed++;
  }

  final public int getSpeed() {
     return speed;
  }
}

public class SportsCar extends Car {
  private boolean turbo;

  public SportsCar() {
    super(120);
    turbo = true;
  }
}
```

Ejemplo de herencia

carrera

```
public class Car {
  public int speed;
  private int wheels = 4;
  private int persons = 0;

  //El método private no se hereda
  private void changeTire() {
     wheels--;
  }

  //El método protected se hereda
  protected void brake() {
    speed = 0;
  }

  //Un método friend se puede llamar si
  //el usuario está en el paquete
  void acelerate() {
    speed++;
  }

  //El método public se hereda
  public void openDoor() {
     persons++;
  }
}
```

```
//La clase SportsCar deriva de Car
public class SportsCar extends Car {
  private int injectors = 12;

  public void turbo() {
    wheels+;
    changeTire();
    speed--;
    brake();
    openDoor();
    injectors++;
    acelerate();
  }
}
```

```
public class Carrera {
  public static void main(String [] arg){
    SportsCar d = new SportsCar();
    d.inyectores++;
    d.changeTire();
    d.speed = 5;
    d.openDoor();
    d.turbo();
  }
}
```

Ejemplo de herencia

carrera

```java
public class Car {
  public int speed; //Mal estilo
  private int wheels = 4;
  private int persons = 0;

  //El método private no se hereda
  private void changeTire() {
     wheels--;
  }

  //El método protected se hereda
  protected void brake() {
    speed = 0;
  }

  //Un método friend se puede llamar si
  //el usuario está en el paquete
  void acelerate() {
    speed++;
  }

  //El método public se hereda
  public void openDoor() {
     persons++;
  }
}
```

```java
//La clase SportsCar deriva de Car
public class SportsCar extends Car {
  private int injectors = 12;

  public void turbo() {
    wheels+;  //Error, no visible
    changeTire(); //Error, no visible
    speed--;  //Mal estilo
    brake();
    openDoor();
    injectors++;
    acelerate();
  }
}
```

```java
public class Carrera {
  public static void main(String [] arg){
    SportsCar d = new SportsCar();
    d.inyectores++; //Error, no visible
    d.changeTire(); //Error, no visible
    d.speed = 5; //Mal estilo
    d.openDoor();
    d.turbo();
  }
}
```

Polimorfismo dinámico

El polimorfismo es una potente herramienta que permite utilizar jerarquías de clases sin preocuparse de los detalles específicos de cada clase.

Referencia de un tipo, objeto de otro



En Java un objeto de un tipo T puede estar referenciado por una referencia del tipo T o por una referencia de cualquier tipo P padre de T.

```
class P {
}

class T extends P {
}

...

externalMethod() {
  T t = new T();
  P p;
  p = t;
}
```

```
class Car {
}

class SportsCar extends Car {
}

...

externalMethod() {
  SportsCar d = new SportsCar();
  Car c;
  c = d;
}
```

Polimorfismo dinámico

Se ejecuta el método del objeto creado



Cuando se invoca un método de un objeto de tipo T, se ejecuta el método de T, sin importar el tipo que tenga la referencia.

```
public class Car {
    public void acelerate() {
        System.println("Run");
    }
}
```

```
public class SportsCar extends Car {
    void acelerate() {
        System.println("Brum, brum");
    }
}
```

```
...
externalMethod() {
  SportsCar d = new SportsCar();
  Car c;
  c = d;
  c.acelerate();
  d.acelerate();
}
```

Polimorfismo dinámico

Se ejecuta el método del objeto creado



Cuando se invoca un método de un objeto de tipo T, se ejecuta el método de T, sin importar el tipo que tenga la referencia.

```
public class Car {
    public void acelerate() {
        System.println("Run");
    }
}
```

```
public class SportsCar extends Car {
    void acelerate() {
        System.println("Brum, brum");
    }
}
```

```
...
externalMethod() {
  SportsCar d = new SportsCar();
  Car c;
  c = d;
  c.acelerate(); //Imprime "Brum, brum" pues el objeto creado es de tipo SportsCar
  d.acelerate();  //Imprime "Brum, brum" pues el objeto creado es de tipo SportsCar
}
```

Polimorfismo dinámico

El tipo de la referencia restringe



Sobre una referencia de tipo P solo se pueden invocar los métodos que tiene P.

```
public class Car {
    void acelerate() {
        System.println("Run");
    }
}
```

```
public class SportsCar extends Car {
    public void acelerate() {
        turbo();
    }
    public void turbo() {
        System.println("Brum, brum");
    }
}
```

```
...

externalMethod() {
  Car c = new SportsCar();
  c.acelerate();
  c.turbo();
}
```

Polimorfismo dinámico

El tipo de la referencia restringe



Sobre una referencia de tipo P solo se pueden invocar los métodos que tiene P.

```
public class Car {
    void acelerate() {
        System.println("Run");
    }
}
```

```
public class SportsCar extends Car {
    public void acelerate() {
        turbo();
    }
    public void turbo() {
        System.println("Brum, brum");
    }
}
```

```
...

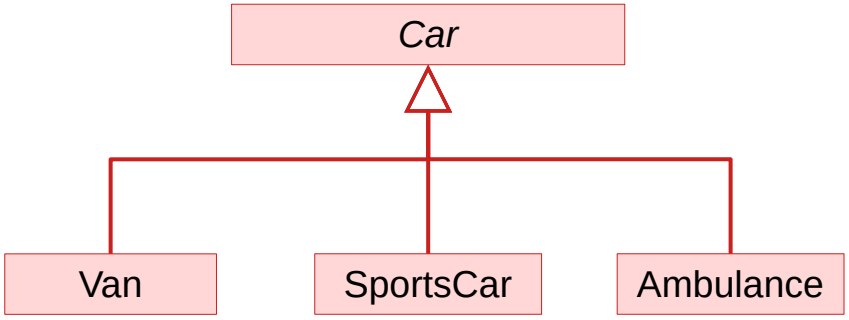
externalMethod() {
  Car c = new SportsCar();
  c.acelerate(); //Imprime "Brum, brum" pues el objeto creado es de tipo SportsCar
  c.turbo();  //Da error en comStackción, porque el tipo Car no tiene método turbo
}
```

Polimorfismo dinámico

El nombre de polimorfismo

Una cosa es polimorfa si tiene múltiples formas.

En el ejemplo de la izquierda, la referencia c a Car es polimorfa porque en ejecución (dinámicamente) puede tomar la forma de un Van, una Ambulance, un SportsCar...




```java
public class Car {
}
```

```java
public class Ambulance extends Car {
}
```

```java
public class Van extends Car {
}
```

```java
public class SportsCar extends Car {
}
```

```java
public class PolimorficExample{

  public static Car getCar() {
    final double azar = Math.random();

    if (azar > 0.75)
      return new Ambulance();
    else if (azar > 0.5)
      return new Van();
    else
      return new SportsCar();
  }

  public static void main(String [] args) {
    Car c = getCar();
    //¿De qué tipo es lo que apunta c?
  }
}
```

Polimorfismo dinámico

¿Qué es el polimorfismo?

El polimorfismo se refiere a la propiedad por la que es posible enviar mensajes sintácticamente iguales a objetos de tipos distintos.

Gracias a la posibilidad de usar una referencia de un tipo base para apuntar a tipos derivados se consigue el polimorfismo dinámico en Java.

```
class A {
  public void printName() {
    System.out.println("A")
  }
}

class B extends A {
  public void printName() {
    System.out.println("B")
  }
}

class C extends A {
  public void printName() {
    System.out.println("C")
  }
}
```

```
A a;

if (Math.random() > 0.5)
     a = new B();
else
     a = new C();

a.printName(); //¿Qué método se invocará?
```

Polimorfismo dinámico

Ejemplo de uso de polimorfísmo dinámico

```java
public class Car {
  protected int speed;

  public void acelerate() {
    System.out.println("Puf, puf...");
    speed += 1;
  }
}
```

```java
public class Ambulance extends Car {
  public void acelerate() {
    System.out.println("Ninoninonino...");
    speed += 10;
  }
}
```

```java
public class SportsCar extends Car {
  public void acelerate() {
    System.out.println("Fiuuuu");
    speed += 20;
  }
}
```

```java
public class Van extends Car {
  private boolean estaloaddo = false;

  public Van(boolean loaddo) {
    this.loaddo = loaddo;
  }

  public void acelerate() {
    if (estaloaddo)
      System.out.println("Bruuumm");
    super.acelerate();
  }
}
```

```java
public class EjemploPolimorfismo{

  public static Car getCar() {
    final double randNumber = Math.random();

    if (azar > 0.75)
      return new Ambulance();
    else if (randNumber > 0.5)
      return new Van(true);
    else if (randNumber > 0.25)
      return new Car();
    else
      return new SportsCar();
  }

  public static void main(String [] args) {
    Car c = getCar();
    c.acelerate();
  }
}
```

La llamada a c.acelerate() es idéntica para todas las clases derivadas de Car.

Solo en ejecución se puede saber cuál de las formas se ejecutará.

Polimorfismo dinámico

Otro ejemplo de uso de polimorfismo

```
public class Car {
  private int speed;
  public void acelerate() {
    speed++;
  }
  public void brake() {
    speed--;
  }
  public int getSpeed() {
    return speed;
  }
}
```

```
public class Ambulacia extends Car {
  public void acelerate() {
    System.out.println("Ninoninonino...");
    for (int c = 0; c < 10; c++)
      super.acelerate();
  }
}
```

```
public class SportsCar extends Car {
  public void acelerate() {
    for (int c = 0; c < 20; c++)
      super.acelerate();
  }
  public void brake() {
    speed-=2;
  }
}
```

```
public class ZonaDeFrenado {

  public stopCar(Car c) {

    while (c.getSpeed() > 0)
      c.brake();
  }
}
```

Puedo llamar al método `stopCar` de `ZonaDeFrenado` pasándole cualquier tipo de `Car`.

La herencia se usa para particularizar

La herencia suele usarse para particularizar la funcionalidad de una clase de la que se hereda.

Por ejemplo: Se hereda de una clase `Text` y se crea la clase `CorrectText` que sobreescribe el método `insertWord` para evitar que se inserten palabras incorrectas. Esto lo hace llamando al insertWord del padre solo cuando la palabra insertada está en un diccionario.

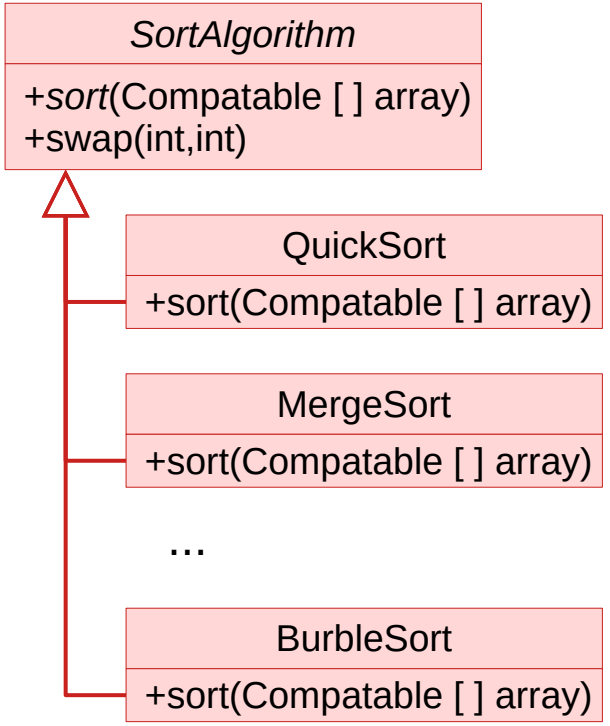
En este caso , además de evitar copiar y pegar código al crear clases similares y evitar tocar una clase que funciona, gracias al polimorfismo se posibilita que un cliente que use objetos de tipo `Text` use objetos de la clase `CorrectText` sin saberlo.

Polimorfismo dinámico

Organizando familias de algoritmos

La herencia se usa para crear familias de clases que ejecutan algoritmos parecidos. Por ejemplo: se hereda de `SortAlgorithm` para crear las clases `QuickSort`, `MergeSort`...

La herencia permitiría que se compartan métodos comunes entre las diferentes clases que heredan de la clase padre sin duplicar código. Así, en el ejemplo, la clase padre podría tener métodos comunes para insertar elementos, intercambiarlos...



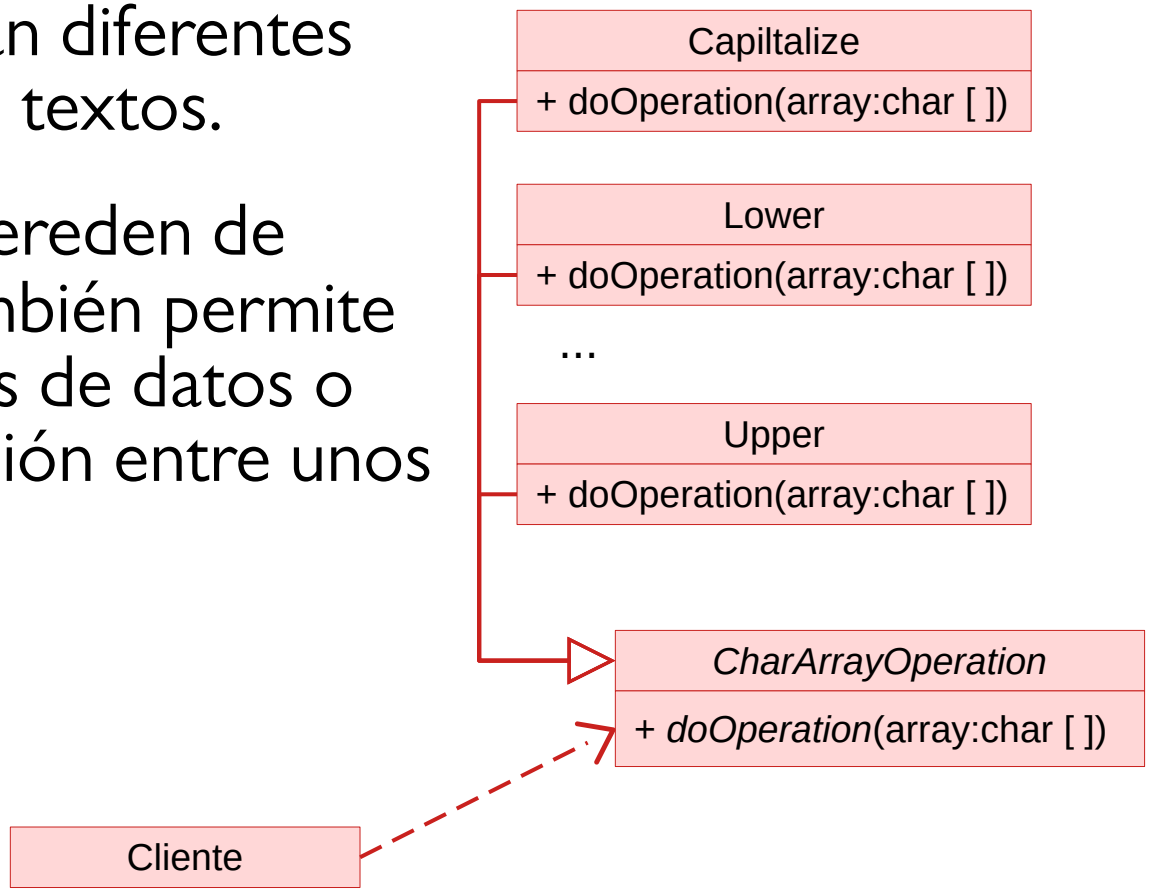
Además, de nuevo gracias al polimorfismo, facilita a los usuarios aprender a usar los diferentes algoritmos, al dotarlos de la misma interfaz.

Polimorfismo dinámico

Usando polimorificamente algoritmos similares

Otro ejemplo sería crear una clase `CharArrayOperation` de la que hereden otras que realizarán diferentes tipos de operaciones sobre textos.

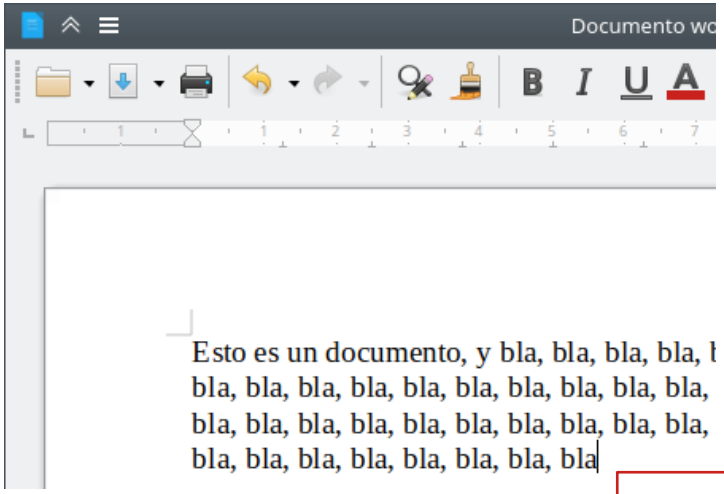
Que los diferentes filtros hereden de `CharArrayOperation` también permite almacenarlos en estructuras de datos o ejecutarlos sin hacer distinción entre unos y otros.

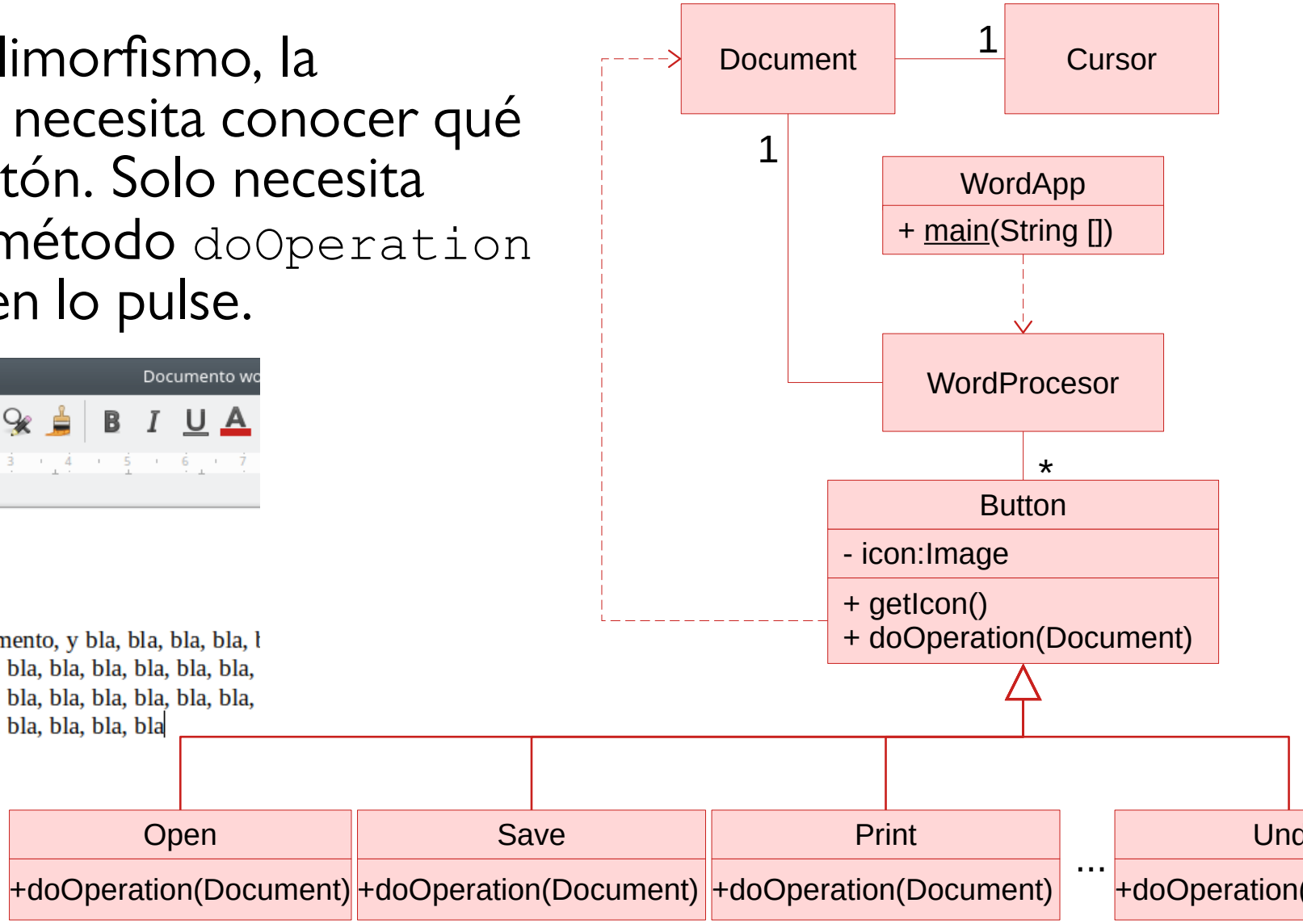


Polimorfismo dinámico

Uso de polimorfismo en un proyecto real

Gracias al polimorfismo, la aplicación no necesita conocer qué hace cada botón. Solo necesita invocar a su método `doOperation` cuando alguien lo pulse.





Consecuencias de la jerarquía única en Java

Se ha visto que:

- En Java un objeto de un tipo T puede estar referenciado por una referencia del tipo T o por una referencia de cualquier tipo padre de T.
- Todas las clases de Java derivan, en última instancia, de la clase `Object`.

Por tanto, cualquier objeto de Java puede ser apuntado por una referencia de tipo `Object`.

Entre otras cosas, esto permite crear estructuras de datos que admiten cualquier clase de objeto.

```
public class Car {
  public void acelerate() {
    ...
  }
}
```

```
public class Stack{

  public void push(Object o) {
    ...
  }

  public Object pop() {
    ...
  }

  public boolean isEmpty() {
    ...
  }
}
```

```
externalMethod() {
  Stack p = new Stack();
  Car c = new Car();
  p.push(c);
  ...
  Object o = p.pop();
  //o.acelerate() no comStackría

  Car c2 = (Car) o;
}
```

Polimorfismo dinámico

El casting de clases

En ocasiones, se dispone de una referencia a un objeto de un tipo T, pero se sabe que dicha referencia en realidad apunta a un objeto de un tipo D derivado de T. Si se necesita, Java permite reasignar dicho objeto a una referencia de D mediante un casting.

- Un objeto referenciado desde un tipo base se puede reasignar a un tipo derivado haciendo casting. Si el casting falla, porque el objeto no es del tipo derivado, se producirá un error durante la ejecución del programa.
- Para evitar el error de casting, se puede comprobar el tipo del objeto usando la palabra reservada `instanceof`.
- En general, el uso correcto del polimorfismo evita los castings y las llamadas a `instanceof`.

```
public class Car {
}
```

```
public class SportsCar extends Car {
}
```

```
externalMethod() {
  SportsCar d = new SportsCar();
  Car c = d;
  if (c instanceof SportsCar)
    SportsCar d2 = (SportsCar) c;
}
```

Clases abstractas e interfaces

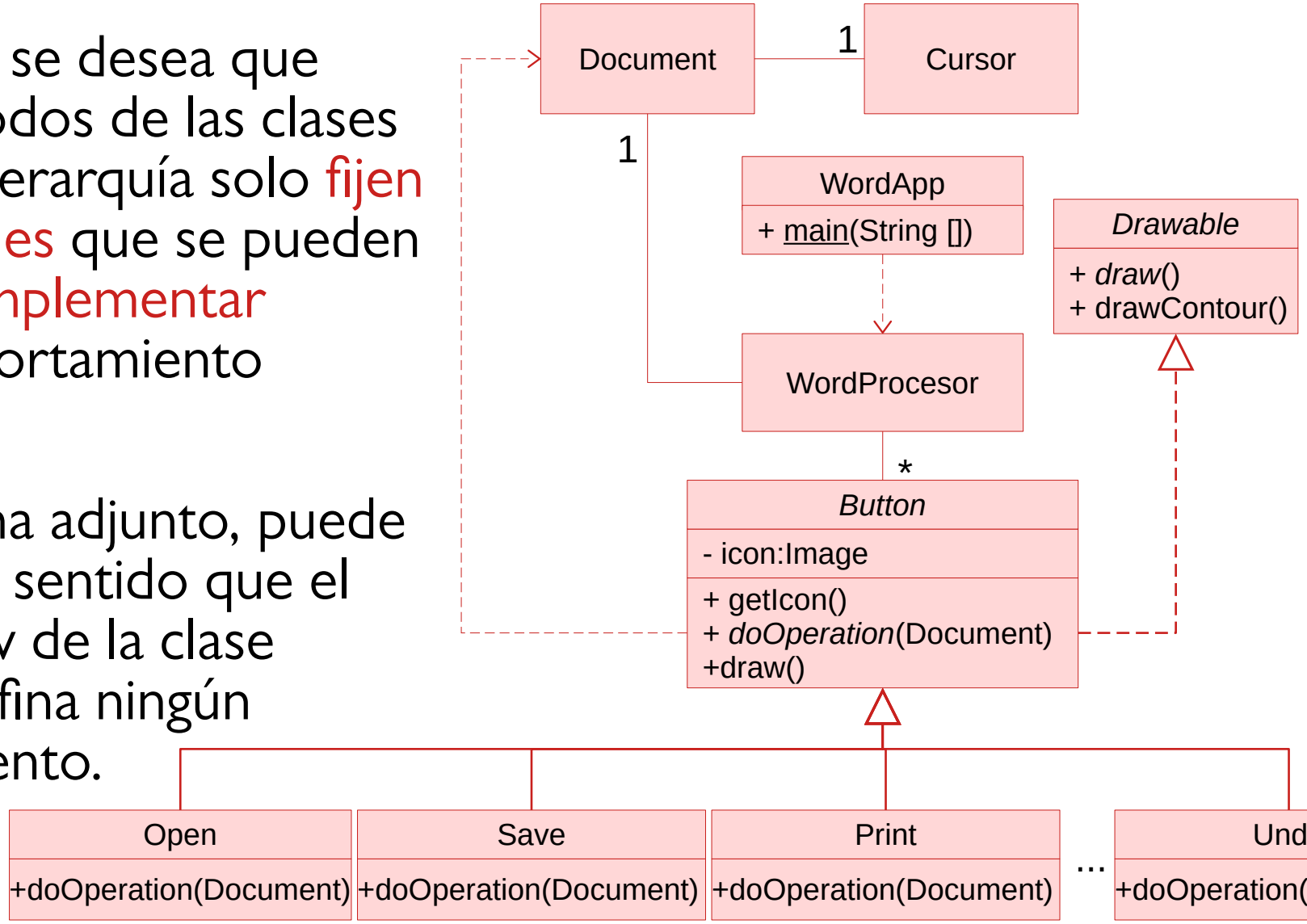
En esta sección se analiza la separación entre implementación e interfaz que subyace al lenguaje Java.

Clases abstractas e interfaces

Necesidad de métodos sin implementación

En ocasiones se desea que algunos métodos de las clases base de una jerarquía solo fijen las operaciones que se pueden realizar, sin implementar ningún comportamiento concreto.

En el diagrama adjunto, puede que no tenga sentido que el método draw de la clase Drawable defina ningún comportamiento.



Las clases abstractas

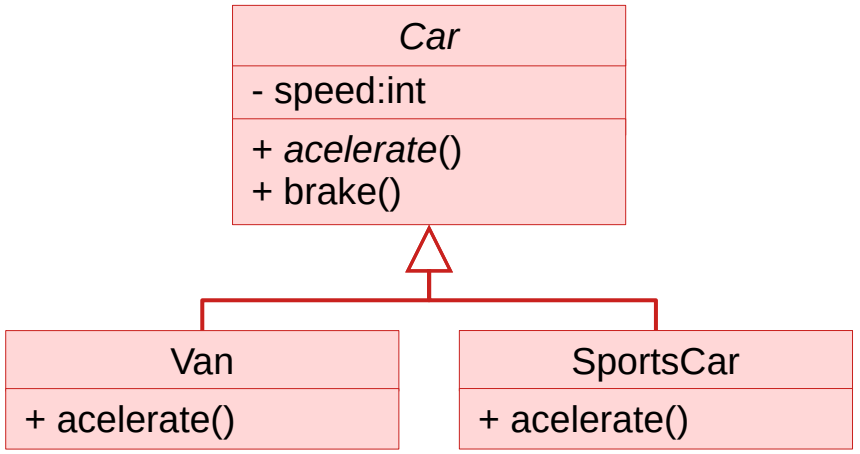


- Las clases abstractas se usan para agrupar comportamiento común de las clases derivadas.
- En UML se representa lo abstracto poniéndolo en cursivas.
- El modificador `abstract` indica los métodos que no tienen implementación.
- Una clase con métodos abstractos se dice que es una clase abstracta.

```
public abstract class Car {
  private int speed;

  public abstract void acelerate();

  public final void brake() {
    speed = 0;
  }
}
```



Métodos y clases abstractas en Java

- No pueden crearse objetos de clases abstractas, porque tendría métodos sin implementación y su ejecución daría error.
- Si una clase deriva de clase abstracta debe implementar todos los métodos o indicar que, a su vez, es abstracta.
- Si todos los métodos de una clase abstracta son abstractos (no tienen implementación) se dice que la clase es abstracta pura.

Clases abstractas e interfaces

Ejemplo de uso de métodos abstractos

```java
public abstract class Car {
  protected int speed;

  public abstract void acelerate();
}
```

```java
public class Ambulance extends Car {
  public void acelerate() {
    System.out.println("Ninoninonino...");
    speed += 10;
  }
}
```

```java
public class SportsCar extends Car {
  public void acelerate() {
    System.out.println("Fiuuuu");
    speed += 20;
  }
}
```

```java
public class Van extends Car {
  private boolean estaloaddo = false;

  public Van(boolean loaddo) {
    this.loaddo = loaddo;
  }

  public void acelerate() {
    if (estaloaddo)
      System.out.println("Bruuumm");
    speed += 5;
  }
}
```

```java
public class EjemploPolimorfismo{

  public static Car getCar() {
    final double azar = Math.random();

    if (azar > 0.75)
      return new Ambulance();
    else if (azar > 0.5)
      return new Van(true);
    else
      return new SportsCar();
  }

  public static void main(String [] args) {
    Car c = getCar();
    c.acelerate();
  }
}
```

El método `acelerate` de la clase `Car` se deja abstracto.

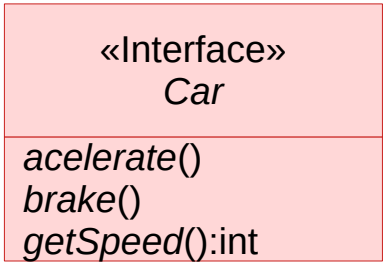
Las clases que derivan de ella se encargan de implementarlo.

Clases abstractas e interfaces

Interfaz



- Una interfaz es una clase abstracta pura y sin propiedades.
- En Java se definen interfaces mediante la palabra reservada `interface`.
- En una interfaz, todos los métodos son públicos por defecto.
- Para que una interfaz herede de otra interfaz se usa la palabra `extends`.
- Si una clase extiende una interfaz debe implementar todos los métodos o indicar que es abstracta.
- En UML las interfaces se distinguen por el estereotipo interface.




```
public interface Car {
  void acelerate();

  void brake();

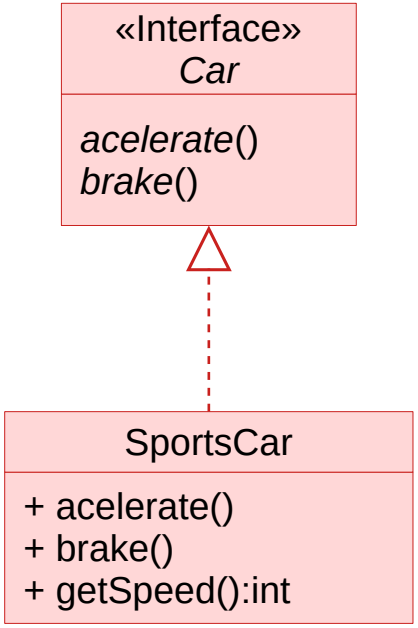
  int getSpeed();
}
```

Implementación de interfaces



En Java se indica que una clase implementa una o varias interfaces utilizando la palabra reservada `implements`.

En UML la herencia de interfaz se representa con una flecha de punta hueca y línea punteada.




```
public interface Car {
  void acelerate();
  void brake();
}
```

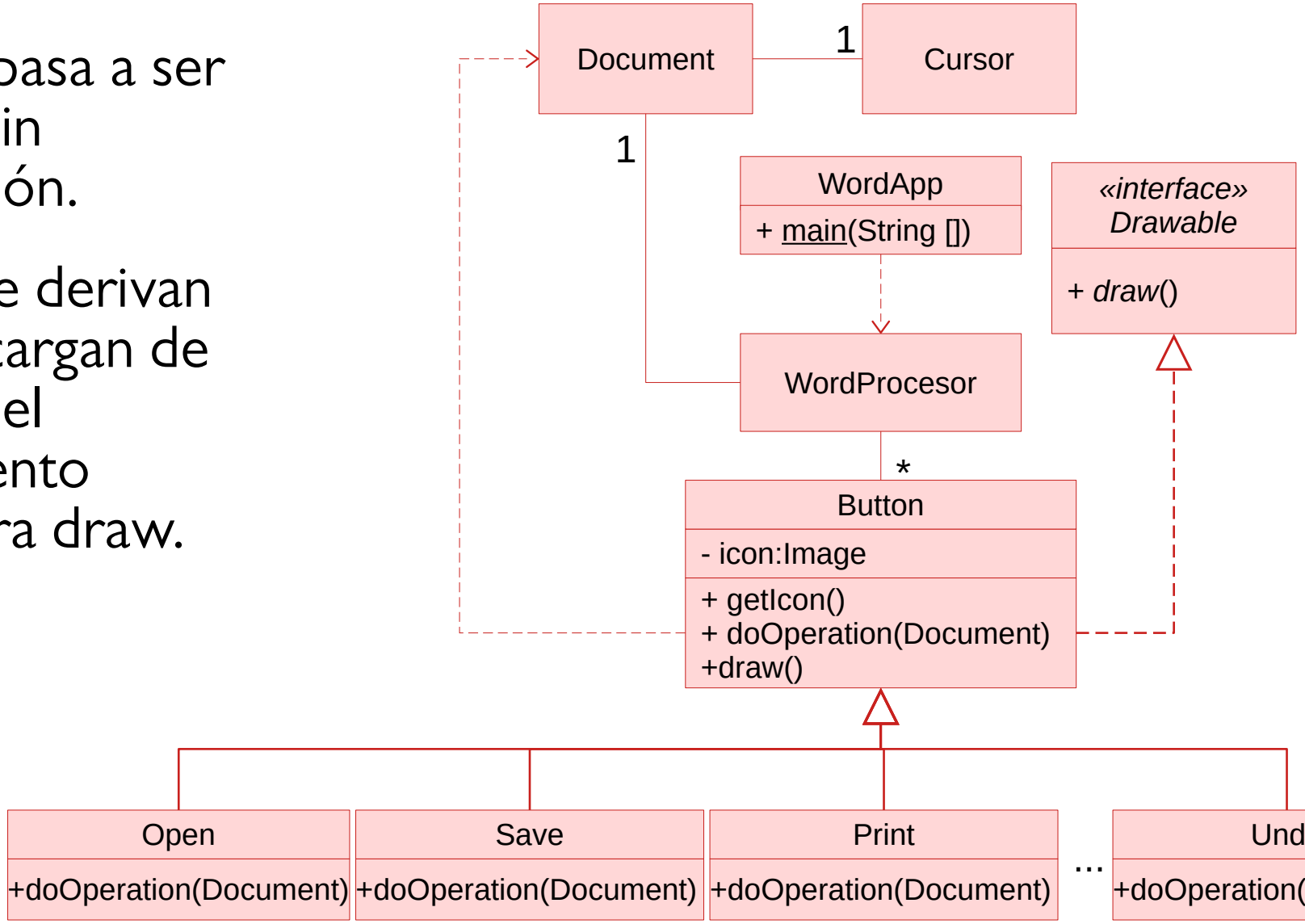
```
public class SportsCar implements Car{
  private int speed = 0;
  public void acelerate() {
    speed++;
  }
  public void brake() {
    speed = 0;
  }
  public int getSpeed() {
    return speed;
  }
}
```

Clases abstractas e interfaces

Ejemplo de uso de interfaces

`Drawable` pasa a ser una interfaz sin implementación.

Las clases que derivan de ella se encargan de implementar el comportamiento adecuado para draw.



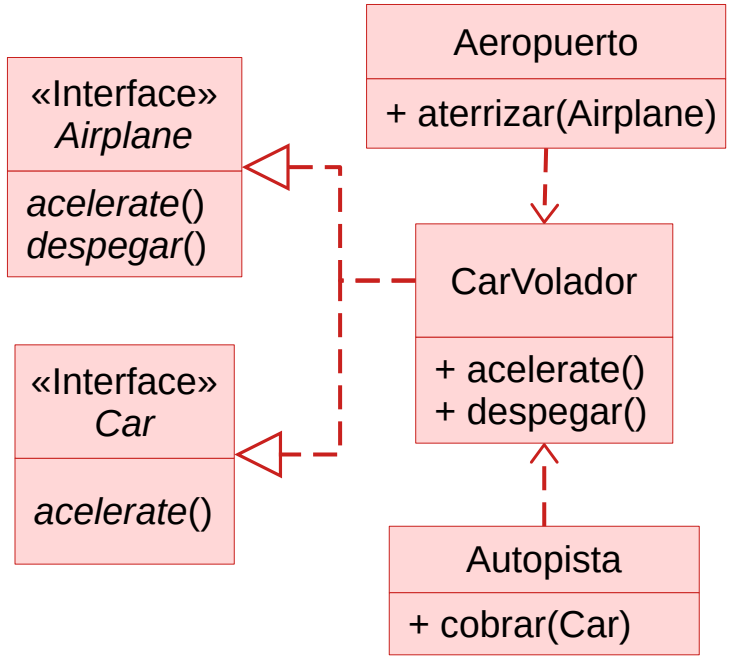
Herencia múltiple de interfaz

Java permite que una clase implemente más de una interfaz.

Que una clase implemente más de una interfaz permite a diferentes tipos de clientes utilizar la clase.

El problema del diamante no aparece porque solo hay una implementación.

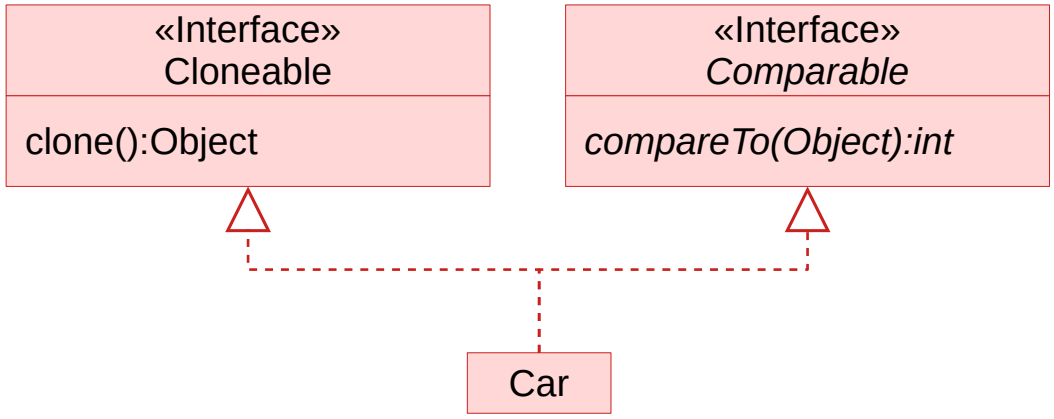
```
public class CarVolador implements Car, Airplane{
  private int speed = 0;
  private int altura = 0;
  public void acelerate() {
    speed++;
  }
  public void despegar() {
    altura++;
  }
}
```



Uso real de herencia múltiple de interfaz

Imaginemos una interfaz `Cloneable` que define a los objetos que se pueden clonar y otra interfaz `Comparable` que define a los objetos que se pueden comparar.

Al programador de la clase `Car` le interesa poder comparar un `Car` con otros y también le interesa poder sacar copias de un `Car`. Por ello decide que `Car` implemente ambas interfaces.



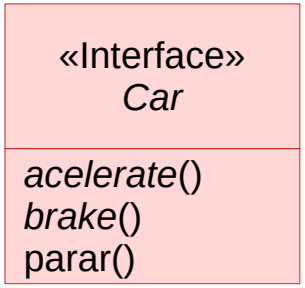
Métodos por defecto en las interfaces



Desde JDK 8 en las interfaces se permite implementar métodos usando el modificador `default`.

Esto se hizo para permitir añadir un método a una interfaz y que se mantenga la compatibilidad hacia atrás.

En el ejemplo de `Car` se introduce el método `stopCar()` y las clases derivadas de `Car` no tienen que ser modificadas gracias a que dicho método tiene una implementación por defecto.




```
public interface Car {
  void acelerate();

  void brake();

  int getSpeed();

  default void stopCar() {
    while (getSpeed > 0)
      brake();
  }
}
```

Problema del diamante en las interfaces



La inclusión de implementación en la interfaces, unida a la posibilidad de herencia múltiple de interfaces conduce de nuevo al problema del diamante.

Java detecta el problema del diamante y fuerza al programador a que lo resuelva implementando el método conflictivo en la clase derivada.

```
interface Airplane{
  default void acelerate() {
    System.out.println("Ziuuuu...");
  }
}

interface Car{
  default void acelerate() {
    System.out.println("Puf, puf...");
  }
}

public class FlyingCar implements Car, Airplane{
```



types Coche and Avion are incompatible;
class CocheVolador inherits unrelated defaults for acelerar() from types Coche and Avion
---
(Alt-Enter shows hints)

Referencias

- “El lenguaje de programación Java”, 3ª Edición, Arnold Gosling, Addison Wesley, 2001.
- “Piensa en Java”. Eckel, 2ª Edición, Addison Wesley, 2002.
- “Introducción a la programación orientada a objetos con Java”, C. Thomas Wu, Mc Graw Hill, 2001.